



TECHNICAL ARCHITECTURE & IMPLEMENTATION GUIDE

راهنمای فنی، معماری و ساختار پروژه

شرح کامل نحوه‌ی کار سامانه‌ی بینایی دست از دریافت تصویر دوربین تا استخراج
Landmark، تشخیص Gesture، آموزش مدل سفارشی، ذخیره‌ی داده و نمایش
داشبورد.

vinext / Vite

WebAssembly

MediaPipe Tasks Vision

TypeScript

React 19

CPU WASM

Inference بدون سرور مدل

2 × 21

۲۱ نقطه برای حداکثر دو دست

Client-first

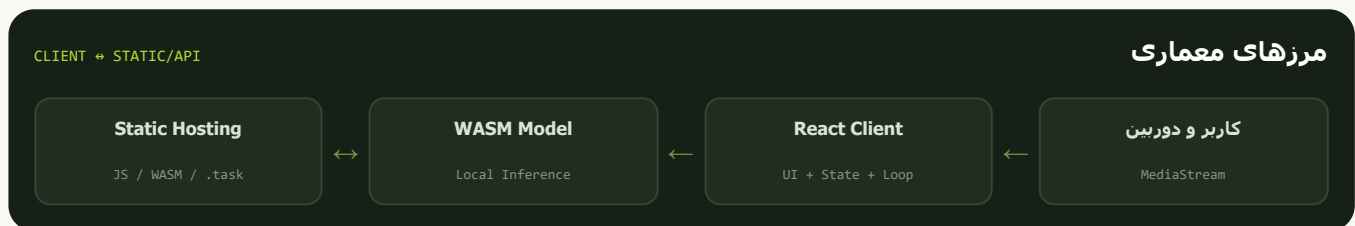
پردازش اصلی داخل مرورگر



نمای کلی سیستم

پروژه یک برنامه‌ی React مبتنی بر vinext است. دوربین، مدل MediaPipe و تحلیل Gesture همگی در Client اجرا می‌شوند؛ سرور فقط فایل‌های برنامه و API سلامت را ارائه می‌کند.

Browser AI



وظیفه‌ی سرور چیست؟

- ارائه‌ی Bundle جاوااسکریپت و CSS
- ارائه‌ی فایل مدل و WASM محلی
- پاسخ API سلامت در /api/health
- میزبانی نسخه‌ی Build شده

چرا Client-first؟

- فریم ویدئو به یک‌اند ارسال نمی‌شود
- Latency شبکه از حلقه‌ی تشخیص حذف می‌شود
- بعد از بارگذاری فایل‌ها، پردازش مستقل است
- حریم خصوصی و هزینه‌ی سرور بهتر می‌شود

نکته‌ی امنیتی

دوربین فقط در Secure Context کار می‌کند. آدرس localhost در توسعه امن محسوب می‌شود؛ نسخه‌ی منتشرشده باید از HTTPS استفاده کند.



چرخه‌ی پردازش زنده

تابع `start()` دوربین و مدل را آماده می‌کند و سپس یک حلقه‌ی `requestAnimationFrame` با FPS قابل تنظیم اجرا می‌شود.

دریافت مجوز و Stream دوربین

با `navigator.mediaDevices.getUserMedia()` Camera، Resolution و Frame Rate انتخاب‌شده فراخوانی می‌شود.

بارگذاری Runtime محلی

به‌صورت `/vision/vision_bundle.mjs` یا `Dynamic Import` و فایل‌های WASM از مسیر `/vision/wasm` بارگذاری می‌شوند.

ساخت Gesture Recognizer

مدل `gesture_recognizer.task` با Delegate پردازنده، حالت VIDEO، حداکثر دو دست و آستانه‌های ۰/۵۵ ساخته می‌شود.

اجرای Inference روی فریم

`recognizeForVideo(video, timestamp)` Landmark، Handedness و `Gesture` پایه را برای هر دست بازمی‌گرداند.

تحلیل تکمیلی

`analyzeHands()` تعداد انگشت، Bounding Box، Pinch، مرکز، سرعت، جهت، Wave، Swipe و مدل سفارشی را محاسبه می‌کند.

فیلتر و ارائه نتیجه

Confidence Threshold و حالت Static/Dynamic اعمال می‌شوند؛ سپس Canvas، UI، تاریخچه و کنترل‌های تعاملی به‌روزرسانی می‌شوند.

جریان یک فریم

FRAME PIPELINE

UI + History

React State

HandSnapshot

Derived Features

RawResult

Landmarks + Scores

Recognizer

WASM / CPU

Video Frame

HTMLVideoElement

Metrics

زمان Inference با `performance.now()` اندازه‌گیری می‌شود؛ FPS از تعداد فریم‌های یک ثانیه‌ی اخیر به‌دست می‌آید.

کنترل نرخ اجرا

فاصله‌ی اجرا برابر `targetFps / 1000` است. فریم‌هایی که زودتر می‌رسند رد می‌شوند تا مصرف CPU کنترل شود.



الگوریتم تحلیل دست و حرکت

خروجی MediaPipe با قواعد هندسی و حافظه‌ی زمانی ترکیب می‌شود تا ویژگی‌هایی فراتر از Gesture پایه ساخته شود.

Pinch

فاصله‌ی نوک شست و اشاره نسبت به اندازه‌ی کف دست نرمال می‌شود. امتیاز نزدیک به ۱ یعنی تماس قوی‌تر.

$$\text{pinch} = \text{clamp}(1 - d(\text{thumb}, \text{index}) / (\text{palm} \times 0.7))$$

شمارش انگشت

برای هر انگشت، فاصله‌ی نوک انگشت از مچ با مفصل میانی مقایسه می‌شود. ضرایب جداگانه، تفاوت ساختار شست و سایر انگشت‌ها را جبران می‌کنند.

$$\text{open} = \text{dist}(\text{tip}, \text{wrist}) > \text{dist}(\text{joint}, \text{wrist}) \times \text{ratio}$$

سرعت و جهت

تا ۳۴ نقطه‌ی مچ در پنجره‌ی ۱۱۰۰ میلی‌ثانیه نگهداری می‌شود. اختلاف اولین و آخرین نقطه سرعت و محور غالب را تعیین می‌کند.

$$\text{speed} = \text{hypot}(\text{dx}, \text{dy}) / \text{elapsed} \times 1000$$

Bounding Box و مرکز

کمینه و بیشینه‌ی مختصات X/Y بین ۲۱ نقطه محاسبه می‌شوند. مرکز کادر برای اشاره‌گر و کنترل حرکتی استفاده می‌شود.

$$\text{center} = ((\text{minX} + \text{maxX}) / 2, (\text{minY} + \text{maxY}) / 2)$$

Swipe

اگر سرعت بیشتر از ۰/۵ باشد، محور غالب X یا Y و علامت جابه‌جایی به Swipe چپ، راست، بالا یا پایین تبدیل می‌شود.

Wave

در ۶۵۰ میلی‌ثانیه‌ی اخیر، تغییر علامت حرکت افقی شمرده می‌شود. حداقل دو برگشت، دامنه‌ی بیشتر از ۰/۱۲ و سه انگشت باز لازم است.

DECISION ORDER

اولویت انتخاب Gesture

Custom

Low Base Confidence

Wave / Swipe

Temporal Override

Pinch / OK

Geometry Override

Gesture پایه

MediaPipe Score

آینه‌ای بودن تصویر

مختصات Canvas هنگام نمایش برای حالت Mirror تبدیل می‌شوند. جهت‌های حرکتی نیز با تجربه‌ی کاربر در تصویر آینه‌ای هماهنگ شده‌اند.

ساختار خروجی هر دست

gesture

نام نهایی

fingerCount

5 تا 0

landmarks[21]

X, Y, Z

handedness

Left / Right

speed / direction

حرکت

pinch

1 تا 0

bbox / center

موقعیت

confidence

امتیاز مدل



مدل Gesture سفارشی

آزمایشگاه فعلی یک طبقه‌بند Centroid سبک برای حالت‌های ثابت دست می‌سازد. ورودی مدل مختصات خام نیست؛ ابتدا نسبت به مچ، اندازه و چرخش نرمال می‌شود.

Pipeline آموزش

LANDMARK CLASSIFIER

Centroid

Per Label Mean

Rotation

Middle Axis Up

Scale

Palm Normalization

Translation

Wrist = Origin

Landmark ۲۱

values 63

آموزش

- نمونه‌ها بر اساس Label گروه‌بندی می‌شوند
- هر کلاس حداقل سه نمونه لازم دارد
- میانگین هر مؤلفه، Centroid کلاس است
- مدل شامل Centroids، Labels و Metadata است
- مدل در localStorage ذخیره می‌شود

نرمال‌سازی

- مچ به مبدأ مختصات منتقل می‌شود
- فاصله مچ تا مفصل ۹ معیار Scale است
- محور کف دست به جهت ثابت می‌چرخد
- ۲۱ نقطه 3×3 محور = بردار ۶۳ تایی
- مقادیر تا سه رقم اعشار گرد می‌شوند

شرط پذیرش

Prediction سفارشی باید Confidence حداقل ۰/۵۵ داشته باشد و زمانی جایگزین Gesture پایه می‌شود که مدل پایه نتیجه‌ای ندارد یا Score آن کمتر از ۰/۶۳ است.

Inference

فاصله‌ی RMS بردار ورودی با همه‌ی Centroidها محاسبه و نزدیک‌ترین کلاس انتخاب می‌شود.

```
distance = sqrt(mean((x - centroid)²))
confidence = clamp(1 - distance / 1.25)
```

معیارهای ارزیابی

معیار	روش محاسبه در پروژه	کاربرد
Accuracy	نسبت پیش‌بینی صحیح به کل نمونه‌ها	نمای کلی عملکرد
Precision	میانگین Macro از $TP / (TP + FP)$	کنترل مثبت کاذب
Recall	میانگین Macro از $TP / (TP + FN)$	کنترل نمونه‌های از دست رفته
F1	میانگین هارمونیک Precision و Recall	تعادل دو معیار
Confusion Matrix	شمارش Actual در برابر Predicted	یافتن کلاس‌های مشابه
Curves	آموزش روی ۱۰ بخش افزایشی Dataset	نمایش روند Accuracy و فاصله

محدودیت آماری

ارزیابی فعلی روی همان Dataset آموزش انجام می‌شود و معیار پژوهشی مستقل نیست. برای Validation واقعی باید Train/Validation/Test جدا و Cross-validation اضافه شود.



معماری رابط و State

app/page.tsx کامپوننت اصلی است. منابع مرورگر در Ref نگهداری می‌شوند و داده‌های قابل نمایش در State قرار می‌گیرند.

State - داده‌ی رابط کاربر

- فاز idle/loading/running/error
- دست‌های شناسایی‌شده و Metrics
- تنظیمات Camera و Threshold
- Custom Model و History، Dataset
- Control Mode و Theme، Tab
- Drawer تنظیمات و Voice، Cursor

Refs - داده‌ی سریع و Mutable

- DOM برای videoRef / canvasRef
- MediaStream برای streamRef
- MediaPipe برای recognizerRef
- motionRef برای Trail‌های زمانی
- timingRef / frameTimesRef برای Metrics
- rafRef برای کنترل حلقه‌ی Frame

REACT DATA FLOW

چرخه‌ی State



داده‌های پایدار

کلید منطقی	محتوا	هدف
settings	Camera، Resolution، FPS، Threshold، نمایش‌ها و حالت مدل	بازیابی تنظیمات کاربر
history	زمان، Gesture، دست، تعداد انگشت و Confidence	داشبورد و CSV
samples	Label، بردار ۶۲ تایی و زمان ثبت	Dataset سفارشی
model	Labelها و زمان آموزش، Centroidها، Labelها	Inference سفارشی
theme	light یا dark	ظاهر رابط

History Control

برای جلوگیری از ثبت تکراری، تشخیص‌ها با فاصله‌ی زمانی وارد تاریخچه می‌شوند. خروجی CSV با BOM فارسی تولید و در مرورگر دانلود می‌شود.

Canvas Overlay

روی ویدئو یک Canvas هم‌اندازه قرار دارد. خطوط Skeleton، نقاط مفصل، Bounding Box، Trail و Cursor در هر Render رسم می‌شوند.

WebMCP Tools

دو ابزار وضعیت پروژه و بازکردن بخش‌های داشبورد ثبت می‌شوند؛ در محیط‌هایی که modelContext ندارند، بی‌اثر هستند.

Voice API

SpeechRecognition یا نسخه‌ی WebKit برای فرمان‌های Start، Stop و Change Mode استفاده می‌شود.



ساختار پوشه‌ها و فایل‌ها

درخت زیر فایل‌های عملیاتی پروژه را نشان می‌دهد. پوشه‌های Build و وابستگی‌ها برای خوانایی حذف شده‌اند.

```
hand-state/
├─ app/
│  ├─ page.tsx           # رابط اصلی، Camera، Loop، Dashboard
│  ├─ hand-vision.ts     # الگوریتم دست، حرکت و مدل سفارشی
│  ├─ globals.css        # Design System، RTL و Responsive
│  ├─ layout.tsx         # Layout و Metadata
│  └─ api/health/route.ts # API سلامت JSON
├─ public/
│  ├─ models/gesture_recognizer.task
│  ├─ vision/
│  │  ├─ vision_bundle.mjs
│  │  └─ wasm/*.wasm + *.js
│  └─ gesture-worker.js   # Worker رزرو/قدیمی؛ مسیر فعلی Worker
├─ favicon.svg
├─ components/ui/        # کتابخانه کامپوننت‌های عمومی
├─ hooks/use-mobile.ts   # موبایل Layout تشخیص
├─ lib/utils.ts          # Utility کلاس‌ها
├─ output/pdf/           # مستندات خروجی
├─ .openai/hosting.json  # Sites شناسه و تنظیم
├─ vite.config.ts        # vinext + Sites + Cloudflare
├─ next.config.ts        # Next تنظیم سازگاری
├─ tsconfig.json         # TypeScript
├─ package.json          # Scripts و Dependencies
└─ package-lock.json     # نسخه دقیق وابستگی‌ها
```

فایل	مسئولیت کلیدی
app/page.tsx	شروع/توقف دوربین، بارگذاری مدل، حلقه‌ی Voice، State، UI، inference، کنترل و Export
app/hand-vision.ts	Typeها، نرمال‌سازی، Centroid، انگشت، Pinch، Wave، Swipe، Labelها
app/globals.css	رنگ‌ها، Dark/Light، Drawer، Charts، Camera Stage، Cards و Breakpointها
public/models/*	مدل MediaPipe محلی؛ علت حذف وابستگی به اینترنت
public/vision/*	Runtime جاوااسکریپت و Binaryهای WebAssembly
vite.config.ts	Build چندمحیطی vinext، پلاگین Sites و Cloudflare Worker



اجرا، Build و عیب‌یابی

نسخه‌ی توسعه روی localhost اجرا می‌شود و Build تولیدی توسط vinext برای Worker آماده می‌شود.

اعتبارسنجی و Build

```
npx tsc --noEmit
npm run build

# Production Worker
npm run start
```

اجرای Development

```
# Node.js 22.13+
npm install
npm run dev

# Browser
http://localhost:3000
```

چک‌لیست عیب‌یابی مدل

WASM URL ✓

باید فایل دریافت /vision/wasm/vision_wasm_internal.wasm باشد.

Browser ✓

Chrome یا Edge به‌روز برای WASM، Camera، Speech پیشنهاد می‌شود.

API Health ✓

/api/health وضعیت Runtime و قابلیت‌ها را بررسی‌گرداند.

Secure Context ✓

صفحه باید روی localhost یا HTTPS باشد.

Camera Permission ✓

مجوز دوربین فعال و دوربین در برنامه‌ی دیگری قفل نباشد.

Model URL ✓

/models/gesture_recognizer.task باید پاسخ ۲۰۰ بدهد.

مسیر پیشنهادی توسعه معماری

خروجی	تغییر معماری	فاز
Dataset حرکتی استاندارد	ثبت آرایه‌ی فریم‌ها همراه زمان و Resampling طول Sequence	Dynamic Dataset
Gesture Sequence Recognition	Python training با LSTM، GRU یا TCN و Split مستقل	Temporal Model
Latency کمتر و مدل حرفه‌ای	Export مدل، Quantization و Runtime شتاب‌گرفته	ONNX/WebGPU
Mouse، Volume و Screenshot سراسری	Electron/Tauri + Native bridge برای OS Input	Desktop
همگام‌سازی Dataset و History	API احراز هویت، Database و Object Storage	Backend
پایداری Release	Unit برای E2E، Geometry دوربین Mock و Benchmark دستگاه‌ها	Testing

نتیجه

معماری فعلی برای Demo حرفه‌ای و اجرای کم‌هزینه در مرورگر مناسب است. برای کنترل واقعی سیستم‌عامل و آموزش Deep Learning، لایه‌های Native و Training Backend باید جداگانه اضافه شوند.